

Data Structures

Introduction

Data Structure is a way to store data elements into the memory (i.e. into the main memory) in an organized manner so that operations like addition, deletion, traversal, searching, sorting etc... can be performed on it efficiently.

Two types of Data Structures

Linear/Basic Data Structures: data elements gets stored into the memory in a linear manner (e.g. sequentially) and hence can be accessed linearly/sequentially.

- Array
- Structure & Union
- Linked List
- Stack
- Queue

Non-linear/Advanced Data Structures: data elements gets stored into the memory in a non-linear manner (e.g. hierarchical) and hence can be accessed non-linearly.

- Tree (Hierarchical)
- Graph
- Hash Table
- Binary Heap

What is a Program and Algorithm?

Program:

A program is a finite set of instructions written in any programming language (like C, C++, Java, Python, Assembly etc...) given to the machine to do specific task.

Algorithm:

An algorithm is a finite set of instructions written in human understandable language (like English), if followed, accomplishes a given task.

An algorithm is a finite set of instructions written in human understandable language (like English) **with some programming constraints, if followed**, accomplishes a given task, such an algorithm also called as **pseudocode**.

An algorithm is a template whereas a program is an implementation of an algorithm.

Analysis of an algorithm

Analysis of an algorithm is a work of determining how much time i.e. computer time and space i.e. computer memory it needs to run to completion.

There are two measures of an analysis of an algorithms:

Time Complexity of an algorithm is the amount of time i.e. computer time required for it to run to completion.

Space Complexity of an algorithm is the amount of space i.e. computer memory required for an algorithm to run to completion.

Asymptotic Analysis: It is a **mathematical way** to calculate time complexity and space complexity of an algorithm without implementing it in any programming language.

- In this type of analysis, analysis can be done on the **basis of basic operation** in that algorithm.

e.g. in searching & sorting algorithms comparison is the basic operation and hence analysis gets done on the basis of no. of comparisons.

Time Complexity

"Best case time complexity": if an algo takes min amount of time to run to completion then it is referred as best case time complexity.

"Worst case time complexity": if an algo takes max amount of time to run to completion then it is referred as worst case time complexity.

"Average case time complexity": if an algo takes neither min nor max amount of time to run to completion then it is referred as an average case time complexity.

"Asymptotic Notations":

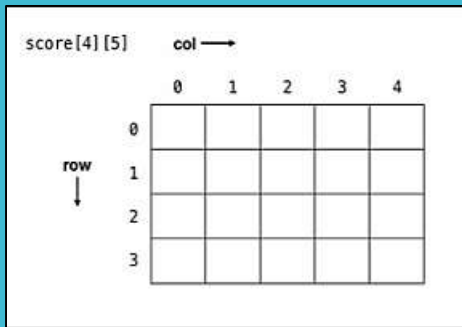
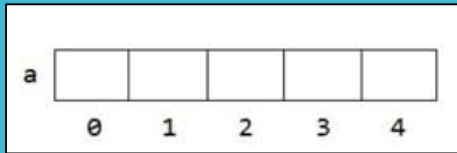
Big Omega (Ω): this notation is used to denote **best case time complexity** – also called as asymptotic lower bound

Big Oh (O): this notation is used to denote **worst case time complexity** - also called as asymptotic upper bound

Big Theta (θ): this notation is used to denote an **average case time complexity** - also called as asymptotic tight bound

Arrays

Array



It is a basic/linear data structure which is a collection/list of logically related similar type of elements in which data elements gets stored into the memory at contiguous locations.

Importance of Arrays in Programming

Arrays are fundamental in programming because they provide efficient means to store and access sequences of data elements with minimal overhead.

Arrays can be classified based on their structure and functionality. Below are the most common types of arrays:

1. One-Dimensional Array (1D Array)

A one-dimensional array is a simple linear list of elements, where each element is accessed using a single index. It is the most basic type of array.

2. Multi-Dimensional Array

A multi-dimensional array consists of arrays inside another array. The most common form is a two-dimensional array (2D array), which is structured like a table with rows and columns.

Array Operations

1. Searching Algorithms

Searching algorithms are used to find an element in an array.

Linear Search: Iterates through each element until the target is found.

Binary Search: Efficiently finds an element in a sorted array by repeatedly dividing the search range in half.

Use Case: Finding a specific product in an inventory list.

2. Accessing Elements

Each element in an array is accessed using its index, starting from 0.

Use Case: Retrieving a specific value from the array.

Array Operations

3. Insertion

Adding new elements to an array can be done at different positions:

At the End – Appending an element increases the array size.

At the Beginning – Elements shift to accommodate the new value.

At a Specific Position – Elements are moved to insert a new value.

Use Case: Adding new data dynamically, such as updating a list of items.

4. Deletion

Elements can be removed from different positions:

From the End – Removes the last element.

From the Beginning – Shifts all elements left.

From a Specific Position – Elements shift to fill the gap.

Search in Array

Finding an element in an array can be done using:

- **Linear Search** – Checking each element sequentially.
- **Binary Search** – Searching efficiently in a sorted array.

Linear Search/ Sequential Search:

Step-1: accept key from the user

Step-2: compare the value of key with each array element sequentially by traversing it from the first element till either key is found or maximum till the last element. If key is found then return true otherwise return false.

```
Algorithm LinearSearch( A, size, key){  
  for( index = 1 ; index <= size ; index++ ){  
    if( key == A[ index ] )  
      return true;  
  }  
  return false;  
}
```

Searching Algorithms

Best Case: If key is found at **very first position** in only 1 no. of comparison, then it is considered as a best case and running time of an algorithm in this case is $O(1) \Rightarrow$ and hence time complexity = $\Omega(1)$

Worst Case: If either key is found at **last position** or key does not exist, maximum **n** no. of comparisons takes place, it is considered as a worst case and running time of an algorithm in this case is $O(n) \Rightarrow$ and hence time complexity = **$O(n)$**

Average Case: If key is found at any in between position it is considered as an average case and running time of an algorithm in this case is $O(n/2) \Rightarrow$ and hence time complexity = $\theta(n)$

Rotate Array – Leetcode Problem

<https://leetcode.com/problems/rotate-array/description/?envType=problem-list-v2&envId=array>

Given	nums	1	2	3	4	5	6	7
Step-1	reverse	7	6	5	4	3	2	1
Step-2	reverse 1st K portion (k=3)	5	6	7	4	3	2	1
Step-3	reverse remaining K portion	5	6	7	1	2	3	4

Rotate Array - Solution

```
void reverse(int* nums, int start, int end) {  
    while (start < end) {  
        int temp = nums[start];  
        nums[start] = nums[end];  
        nums[end] = temp;  
        start++;  
        end--;  
    }  
}  
  
void rotate(int* nums, int numsSize, int k) {  
    if (numsSize == 0) return;  
    k %= numsSize;  
    reverse(nums, 0, numsSize - 1);  
    reverse(nums, 0, k - 1);  
    reverse(nums, k, numsSize - 1);  
}
```

Binary Search/ Logarithmic Search:

This algorithm follows **divide-and-conquer approach**.

To apply binary search on an array prerequisite is that array elements must be in a sorted manner.

Step-1 : accept key from the user

Step-2 : calculate mid position of an array by the formula, **mid = (left+right)/2** (by means of calculating mid position big size array has been divided logically into two subarrays, **from left to mid-1 = left subarray & mid+1 to right = right subarray**)

Step-3 : compare the value of key with element which is at mid position. if key matches with element at mid position means key is found and return true.

Binary Search/ Logarithmic Search:

Step-4 : if key do not matches then check, is the value of key is less than element which is at mid position, if yes then go to search key only into the left subarray by skipping whole right subarray otherwise (means of the value of key is greater than element which is at mid position) go to search key only into the right subarray by skipping whole left subarray.

Step-5 : repeat Step-2, Step-3 & Step-4 till either key is found or max till the subarray is valid, if subarray becomes invalid means key is not found and hence return false in this case.

Binary Search/ Algorithm

```
Algorithm BinarySearch(A, n, key) //A is an array of size "n", and key to be search
{
    left = 1;
    right = n;

    while( left <= right )
    {
        //calculate mid position
        mid = (left+right)/2;
        //compare key with an ele which is at mid position
        if( key == A[ mid ] ) //if found return true
            return true;

        //if key is less than mid position element
        if( key < A[ mid ] )
        {
            right = mid-1; //search key only in a left subarray
        }
        else //if key is greater than mid position element
        {
            left = mid+1; //search key only in a right subarray
        }
    } //repeat the above steps either key is not found or max any subarray is valid
    return false;
}
```

Binary Search/ Algorithm

Best Case: if the key is found in very first iteration at mid position in only 1 no. of comparison it is considered as a best case and running time of an algorithm in this case is $O(1) = \Omega(1)$.

Worst Case: if either key is not found or key is found at leaf position it is considered as a worst case and running time of an algorithm in this case is $O(\log n) = O(\log n)$.

Average Case: if key is found at non-leaf position it is considered as an average case and running time of an algorithm in this case is $O(\log n) = \theta(\log n)$.

Sorting Algorithm

Sorting Algorithms

2. Sorting Algorithms

Sorting algorithms arrange array elements in a specific order (ascending or descending).

Bubble Sort: Repeatedly swaps adjacent elements if they are in the wrong order.

Selection Sort: Selects the smallest element and places it in the correct position.

Insertion Sort: Builds a sorted array one element at a time.

Merge Sort: Uses the divide-and-conquer approach to merge sorted subarrays.

Quick Sort: Partitions the array around a pivot and sorts recursively.

Use Case:

Sorting customer names alphabetically.

Sorting Algorithms

1. Selection Sort:

In this algorithm, in **first iteration**, first position gets selected and element which is at **selected position gets compared with all its next position elements**, if selected position element found greater than any other position element then **swapping takes place** and in first iteration **smallest element gets settled at first position**.

In the **second iteration**, second position gets selected and element which is at selected position gets compared with all its next position elements, if selected position element found greater than any other position element then swapping takes place and in second iteration second smallest element gets settled at second position, and so on in maximum $(n-1)$ no. of iterations **all array elements gets arranged in a sorted manner**.

Sorting Algorithms

Iteration-1	Iteration-2	Iteration-3	Iteration-4	Iteration-5
302060501040 012345 sel_pospos	103060502040 012345 sel_pospos	102060503040 012345 sel_pospos	102030605040 012345 sel_pospos	102030406050 012345 sel_pospos
203060501040 012345 sel_pospos	103060502040 012345 sel_pospos	102050603040 012345 sel_pospos	102030506040 012345 sel_pospos	102030405060 012345
203060501040 012345 sel_pospos	103060502040 012345 sel_pospos	102030605040 012345 sel_pospos	102030406050 012345	
203060501040 012345 sel_pospos	102060503040 012345 sel_pospos	102030605040 012345		
103060502040 012345 sel_pospos	102060503040 012345			
103060502040 012345				

Sorting Algorithms

2. Bubble Sort:

In this algorithm, in every iteration elements which are at **two consecutive positions gets compared**, if they are already in order then no need of swapping between them, but if they are not in order i.e. if **Prev position element is greater than its next position element then swapping** takes place, and by this logic in first iteration largest element gets settled at last position, in second iteration second largest element gets settled at second last position and so on, in max $(n-1)$ no. of iterations all elements gets arranged in a sorted manner.

Best Case : $\Omega(n^2)$

Worst Case : $O(n^2)$

Average Case : $\theta(n^2)$

Sorting Algorithms

Iteration-1	Iteration-2	Iteration-3	Iteration-4	Iteration-5
302060501040 012345 pospos+1	203050104060 012345 pospos+1	203010405060 012345 pospos+1	201030405060 012345 pospos+1	102030405060 012345 pospos+1
203060501040 012345 pospos+1	203050104060 012345 pospos+1	203010405060 012345 pospos+1	102030405060 012345 pospos+1	102030405060 012345 pospos+1
203060501040 012345 pospos+1	203050104060 012345 pospos+1	201030405060 012345 pospos+1	102030405060 012345 pospos+1	
203050601040 012345 pospos+1	203010504060 012345 pospos+1	201030405060 012345 pospos+1		
203050106040 012345 pospos+1	203010405060 012345 pospos+1			
203050104060 012345 pospos+1				

Sorting Algorithms

3. Insertion Sort:

In this algorithm, in every iteration one element gets selected as a key element and key element gets inserted into an array at its appropriate position towards its **left-hand side elements** in a such a way that elements which are at left side are arranged in a sorted manner, and so on, in max $(n-1)$ no. of iterations all array elements gets arranged in a sorted manner.

This algorithm works efficiently for **already sorted** input sequence by design and hence running time of an algorithm is $O(n)$ and it is considered as a best case.

Best Case : $\Omega(n)$ - if array elements are already arranged in a sorted manner.
Worst Case : $O(n^2)$
Average Case : $\theta(n^2)$

Sorting Algorithms

4. Merge Sort:

This algorithm follows divide-and-conquer approach.

In this algorithm, **big size array is divided logically into smallest size** (i.e. having size 1) subarrays, as if size of subarray is 1 it is sorted, after dividing array into sorted smallest size subarray's, **subarrays gets merged into one array** step by step in a sorted manner and finally all array elements gets arranged in a sorted manner.

This algorithm works fine for even as well odd input size array.

This algorithm takes extra space to sort array elements, and hence its space complexity is more.

Best Case : $\Omega(n)$ - if array elements are already arranged in a sorted manner.

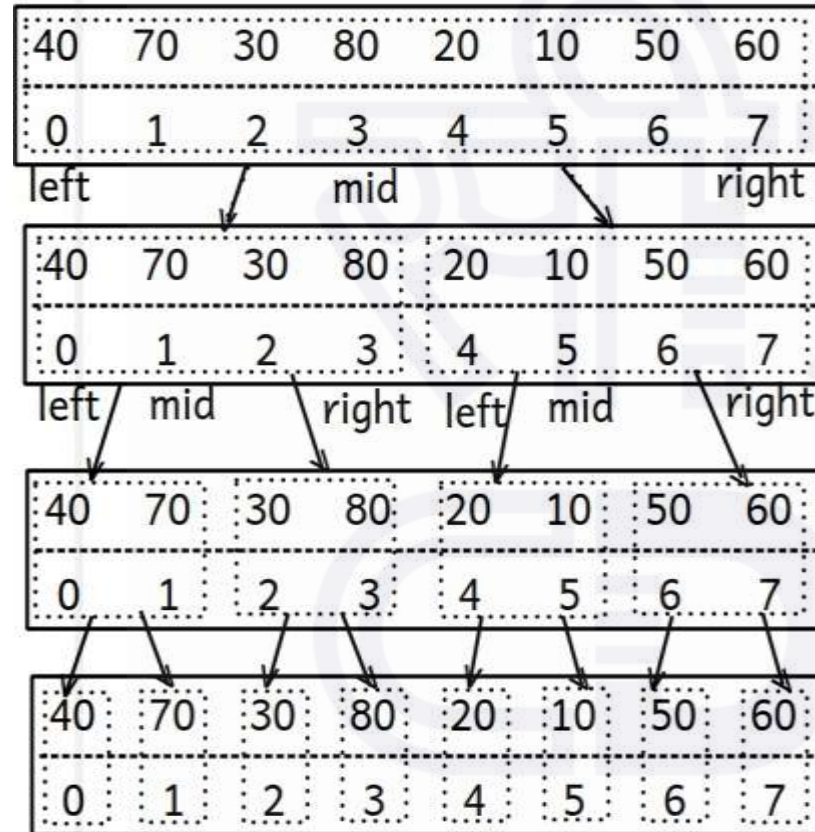
Worst Case : $O(n^2)$

Average Case: $\theta(n^2)$

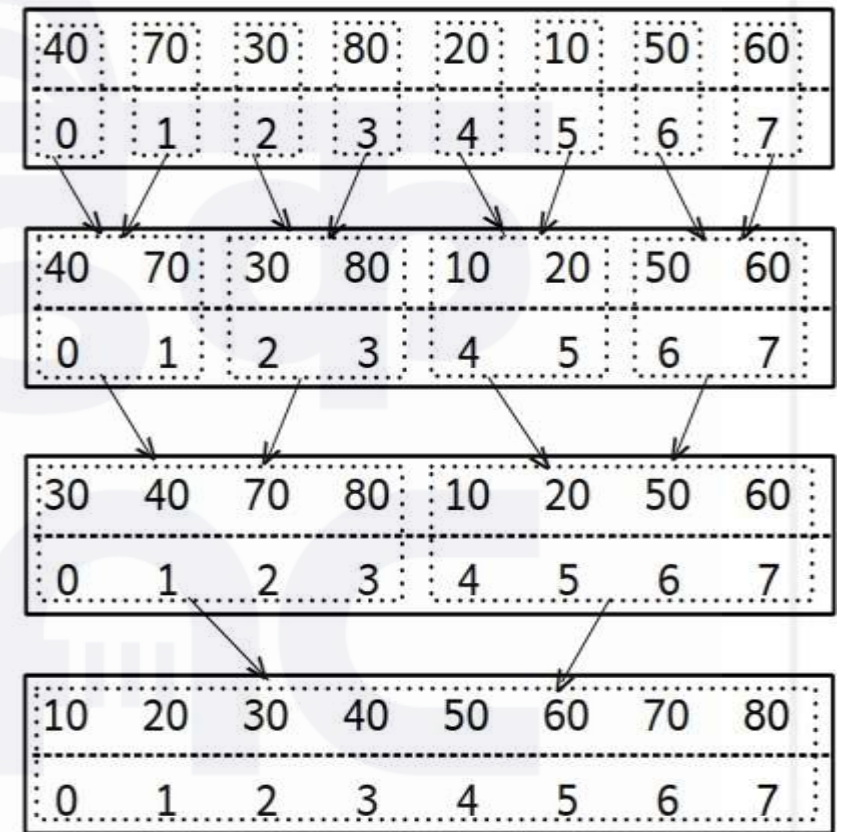
Sorting Algorithms

Merge Sort

Dividing big size array into smallest size subarrays



Merge already sorted arrays



Sorting Algorithms

Quick Sort:

This algorithm follows divide-and-conquer approach.

In this algorithm the basic logic is a partitioning.

Partitioning: in partitioning, pivot element gets selected first (it may be either leftmost or rightmost or middle most element in an array), after **selection of pivot element all the elements which are smaller than pivot gets arranged towards as its left as possible and elements which are greater than pivot gets arranged as its right as possible**, and big size array is divided into two subarrays, so after first pass pivot element gets settled at its appropriate position, elements which are at left of pivot is referred as left partition and elements which are at its right referred as a right partition.

Quick sort algorithm is an efficient sorting algorithm for larger input size array.

Worst Case: $O(n^2)$ – worst case rarely occurs

Best Case : $\Omega(n \log n)$

Worst Case : $O(n \log n)$

Average Case : $\theta(n \log n)$

Linked List

Linked List

Limitations of an array data structure:

Array is static, i.e. size of an array is fixed, its size cannot be either grown or shrink during runtime.

Addition and deletion operations on an array are not efficient as it takes $O(n)$ time, and hence to overcome these two limitations of an Array Linked List data structure has been designed.

Linked List: It is a collection/list of logically related similar type of elements in which, an **address of first element** in a collection/list is stored into a pointer variable referred as a **head pointer**. Each element contains **data and an address of its next** (as well as its previous element).

An element in a Linked List is also called as a **Node**.

Four types of linked lists are there: Singly Linear Linked List, Singly Circular Linked List, Doubly Linear Linked List and Doubly Circular Linked List.

Linked List

Basically, we can perform addition, deletion, traversal etc... operations on linked list data structure.

We can add and delete node by three ways: we can add node into the linked list at last position, at first position and at any specific position, similarly we can delete node from linked list which is at first position, last position and at any specific position.

Singly Linear Linked List: It is a linked list in which head always contains an address of first element, if list is not empty. Each node has two parts:

data part: contains data of any primitive/non-primitive type.

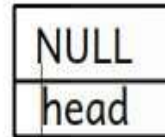
pointer part(next): contains an address of its next element/node.

- last node points to NULL, i.e. next part of last node contains NULL.

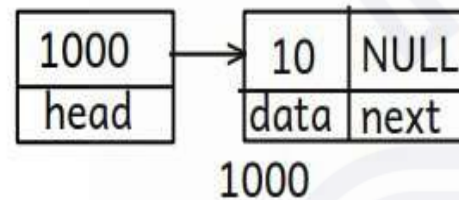
Linked List

SINGLY LINEAR LINKED LIST

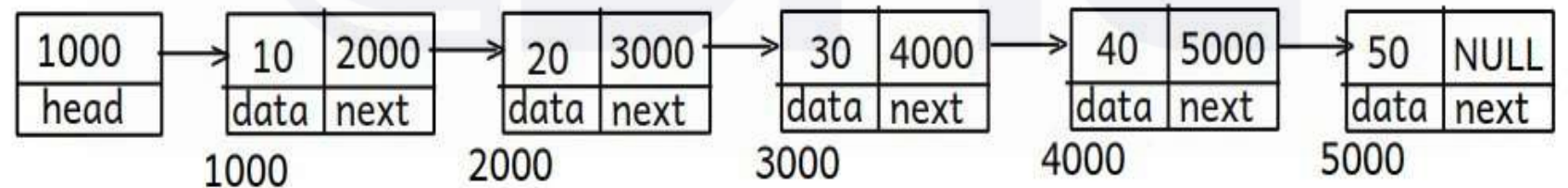
1) singly linear linked list --> list is empty



2) singly linear linked list --> list contains only one node



3) singly linear linked list --> list contains more than one nodes



Linked List

Limitations of Singly Linear Linked List:

Add node at last position & delete node at last position operations are not efficient as it takes $O(n)$ time.

We can start traversal only from first node and can traverse the SLLL only in a forward direction.

Previous node of any node cannot be accessed from it.

Any node cannot be revisited – to overcome this limitation Singly Circular Linked List has been designed.

Singly Circular Linked List: It is a linked list in which head always contains an address of first element, if list is not empty. Each node has two parts:

data part: contains data of any primitive/non-primitive type.

pointer part(next): contains an address of its next element/node.

- last node point to first node, i.e. **next part of last node contains an address of first node.**

Linked List

Limitations of Singly Circular Linked List:

Add last, delete last & add first, delete first operations are not efficient as it takes $O(n)$ time.

We can start traversal only from first node and can traverse the SCLL only in a forward direction.

Previous node of any node cannot be accessed from it – to overcome this limitation Doubly Linear Linked List has been designed.

Doubly Linear Linked List: It is a linked list in which head always contains an address of first element, if list is not empty. Each node has three parts:

data part: contains data of any primitive/non-primitive type.

pointer part(next): contains an address of its next element/node.

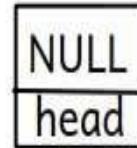
pointer part(prev): contains an address of its previous element/node.

- next part of last node & prev part of first node point to NULL.

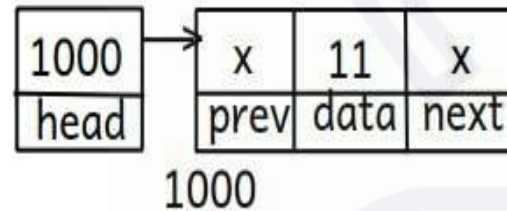
Linked List

DOUBLY LINEAR LINKED LIST

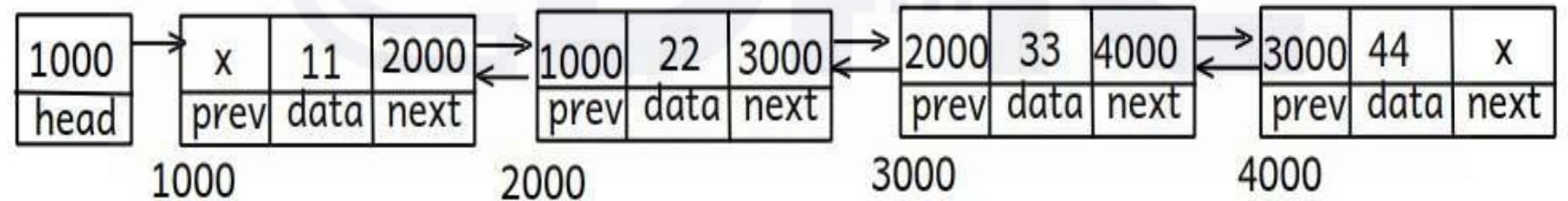
1. doubly linear linked list --> list is empty



2. doubly linear linked list --> list contains only one node



3. doubly linear linked list --> list contains more than one nodes



Linked List

Limitations of Doubly Linear Linked List:

Add last and delete last operations are not efficient as it takes $O(n)$ time.

We can start traversal only from first node, and hence to overcome these limitations Doubly Circular Linked List has been designed.

Doubly Circular Linked List: It is a linked list in which head always contains an address of first node, if list is not empty. Each node has three parts:

data part: contains data of any primitive/non-primitive type.

pointer part(next): contains an address of its next element/node.

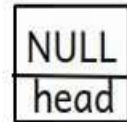
pointer part(prev): contains an address of its previous element/node.

- next part of last node contains an address of first node & prev part of first node contains an address of last node.

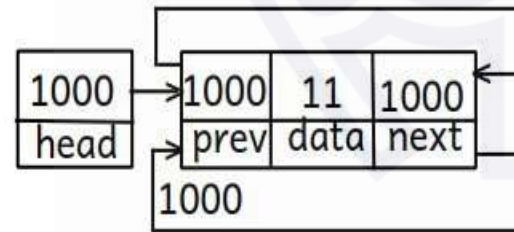
Linked List

DOUBLY CIRCULAR LINKED LIST

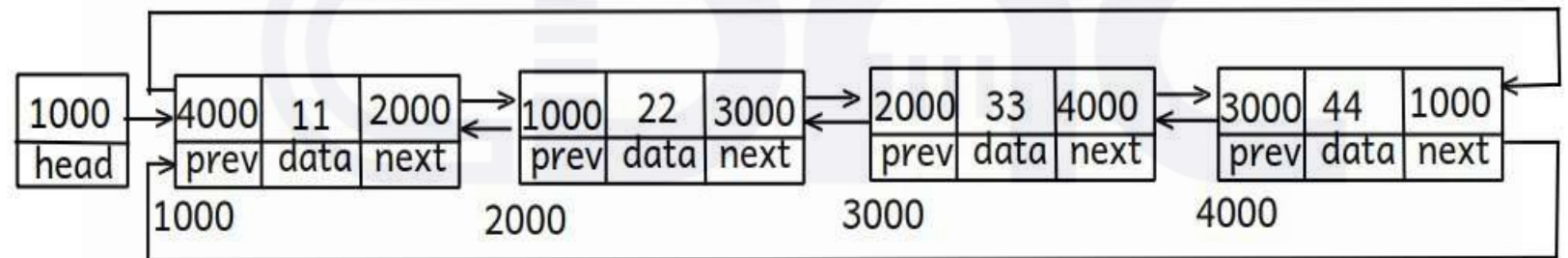
1. doubly circular linked list --> list is empty



2. doubly circular linked list -> list contains only one node



3. doubly circular linked list --> list contains more than one nodes



Linked List

Advantages of Doubly Circular Linked List:

DCLL can be traverse in forward as well as in a backward direction.

Add last, add first, delete last & delete first operations are efficient as it takes $O(1)$ time and are convenient as well.

Traversal can be start either from first node or from last node.

Any node can be revisited.

Previous node of any node can be accessed from it

Array v/s Linked List:

Array is static data structure whereas linked list is dynamic data structure.

Array elements can be accessed by using random access method which is efficient than linked list elements which can be accessed by sequential access method.

Addition & Deletion operations are efficient on linked list than on an array.

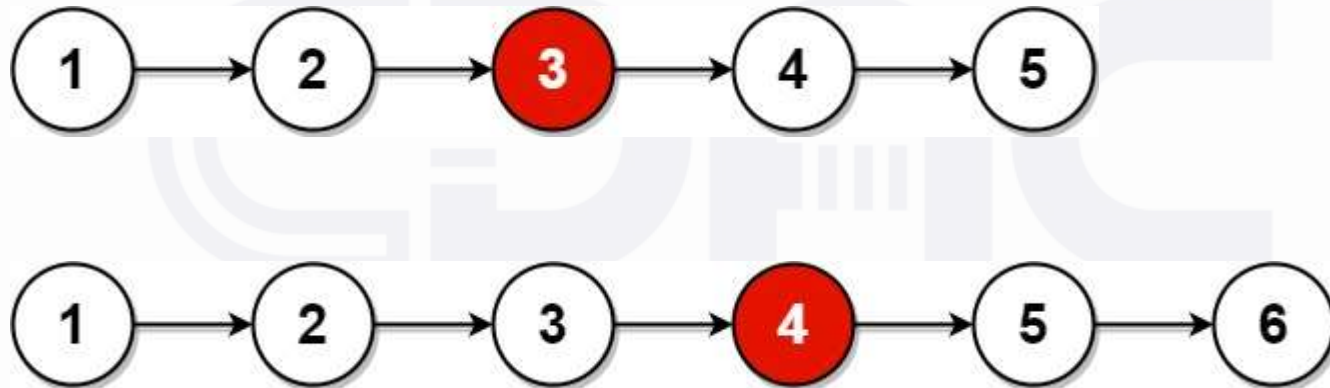
Array elements gets stored into the stack section, whereas linked list elements gets stored into heap section.

In a linked list extra space is required to maintain link between elements, whereas in an array to maintain link between elements is the job of compiler.

Middle of the Linked List – Leetcode Problem

<https://leetcode.com/problems/middle-of-the-linked-list/description/>

- Given the head of a singly linked list, return *the middle node of the linked list*.
- If there are two middle nodes, return **the second middle** node.



Solution

```
/** Definition for singly-linked list.
 * struct ListNode {
 *     int val;
 *     struct ListNode *next;
 * }; */
struct ListNode* middleNode(struct ListNode* head) {
    struct ListNode *slow = head;
    struct ListNode *fast = head;

    while (fast != NULL && fast->next != NULL) {
        slow = slow->next;
        fast = fast->next->next;
    }
    return slow;
}
```

Stack

Stack

Stack: It is a collection/list of logically related similar type elements into which data elements can be added as well as deleted from only one end referred **top end**.

In this collection/list, element which was inserted last only can be deleted first, so this list works in **last in first out/first in last out manner**, and hence it is also called as LIFO list/FILO list.

We can perform basic **three operations** on stack in $O(1)$ time: **Push, Pop & Peek**.

Push : to insert/add an element onto the stack at top position

step1: check stack is not full

step2: increment the value of top by 1

step3: insert an element onto the stack at top position.

Pop: to delete/remove an element from the stack which is at top position

step1: check stack is not empty

step2: decrement the value of top by 1.

Stack

3. Peek: to get the value of an element which is at top position without push & pop.

step1: check stack is not empty

step2: return the value of an element which is at top position

Stack Empty : $\text{top} == -1$

Stack Full : $\text{top} == \text{SIZE}-1$

Applications of Stack:

Stack is used by an OS to control of flow of an execution of program.

In recursion internally an OS uses a stack.

undo & redo functionalities of an OS are implemented by using stack.

Stack is used to implement advanced data structure algorithm like DFS: **Depth First Search traversal** in tree & graph.

Stack is used in an algorithms to covert given infix expression into its equivalent postfix and prefix, and for postfix expression evaluation.

Stack Operations Example

- `stack.push(10);`
- `stack.push(20);`
- `stack.peak();`
- `stack.pop();`
- `stack.push(500);`
- `stack.push(10);`
- `stack.pop();`
- `stack.peak();`

Valid Parentheses Leetcode Problem

<https://leetcode.com/problems/valid-parentheses/description/>

- Given a string `s` containing just the characters '(', ')', '{', '}', '[' and ']', determine if the input string is valid.
- An input string is valid if:
- Open brackets must be closed by the same type of brackets.
- Open brackets must be closed in the correct order.
- Every close bracket has a corresponding open bracket of the same type.
-
- **Example 1:**
- **Input:** `s = "()"`
- **Output:** `true`

Solution

```
bool isValid(char* s) {
    char stack[10000];
    int top = -1;
    for (int i = 0; s[i] != '\0'; i++) {
        char ch = s[i];
        // Opening brackets
        if (ch == '(') {
            stack[++top] = ')'; }
        else if (ch == '{') {
            stack[++top] = '}'; }
        else if (ch == '[') {
            stack[++top] = ']'; }
        // Closing brackets
        else {
            // Stack is empty
            if (top == -1)
                return false;

            // Closing bracket does not match
            if (stack[top] != ch)
                return false;

            // Matching bracket found -> pop
            top--;
        }
    }
    // Valid only if stack is empty
    return top == -1;
}
```

Queue

Queue

Queue: It is a collection/list of logically related similar type of elements into which **elements can be added from one end referred as rear end**, whereas elements can be **deleted from another end referred as a front end**.

In this list, element which was inserted first can be deleted first, so this list works in first in first out manner, hence this list is also called as **FIFO list/LILO list**.

Two basic operations can be performed on queue in $O(1)$ time.

Enqueue: to insert/push/add an element into the queue from rear end.

Dequeue: to delete/remove/pop an element from the queue which is at front end.

There are different types of queue:

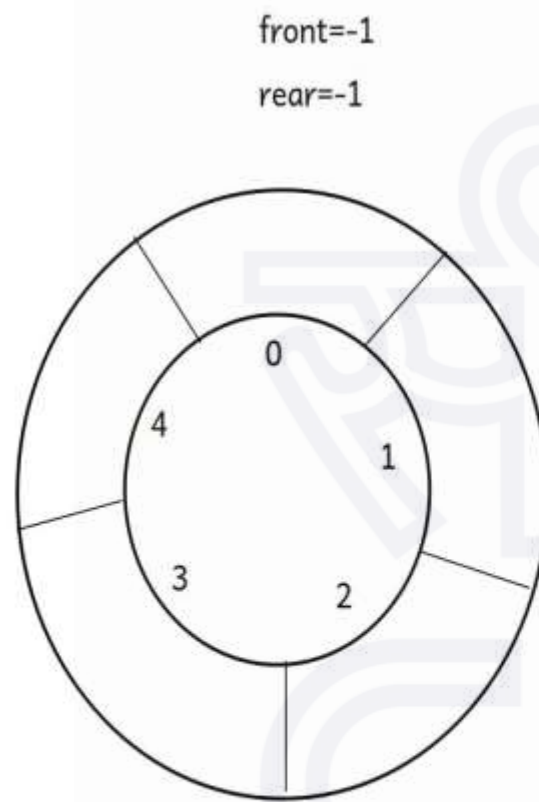
Linear Queue (works in a fifo manner)

Circular Queue (works in a fifo manner)

Priority Queue: it is a type of queue in which elements can be inserted from rear end randomly (i.e. without checking priority), whereas an element which is having highest priority can only be deleted first. Priority queue can be implemented by using linked list, whereas it can be implemented efficiently by using binary heap.

Double Ended Queue (deque) : it is a type of queue in which elements can be added as well as deleted from both the ends.

Queue



Circular Queue

is_queue_full : front == (rear+1)%SIZE

is_queue_empty : rear == -1 && front == rear

1. **"enqueue"**: to insert/add/push an element into the queue from rear end:

step1: check queue is not full

step2: increment the value of rear by 1 [$\text{rear} = (\text{rear} + 1) \% \text{SIZE}$]

step3: push/add/insert an ele into the queue at rear position

step4: if(front == -1)

front = 0

2. **"dequeue"**: to remove/delete/pop an element from the queue which is at front position.

step1: check queue is not empty

step2:

if(front == rear)//if we are deleting last ele

front = rear = -1;

else

increment the value of front by 1 [i.e. we are deleting an ele from the queue]. [$\text{front} = (\text{front} + 1) \% \text{SIZE}$]

Queue Operations Example

- `queue(3);`
- `enqueue(5);`
- `enqueue(10);`
- `dequeue();`
- `peek();`
- `enqueue(20);`
- `enqueue(15);`
- `dequeue();`

Queue

Applications of Queue:

Queue is used to implement OS data structures like **job queue, ready queue, message queue, waiting queue** etc...

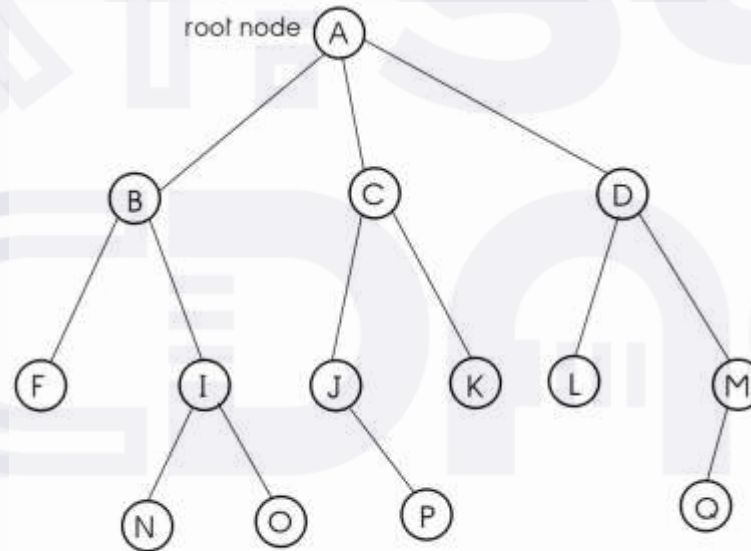
Queue is used to implement OS algorithms like **FCFS CPU Scheduling, Priority CPU Scheduling, FIFO Page Replacement** etc...

Queue is used to implement an advanced data structure algorithms like **BFS: Breadth First Search** Traversal in tree and graph.

Tree

Tree

Tree: It is a non-linear, advanced data structure which is a collection of finite no. of logically related similar type of elements in which, there is a first specially designated element referred as a root element, and remaining all elements are connected to it in a hierarchical manner, follows **parent-child relationship**.



Tree: Data Structure

Tree

siblings/brothers: child nodes of same parent are called as siblings.

ancestors: all the nodes which are in the path from root node to that node.

descendants: all the nodes which can be accessible from that node.

degree of a node = no. of child nodes having that node

degree of a tree = max degree of any node in a given tree

leaf node/external node/terminal node: node which is not having any child node OR node having degree 0.

non-leaf node/internal node/non-terminal node: node which is having any no. of child node/s OR node having non-zero degree.

level of a node = level of its parent node + 1

level of a tree = max level of any node in a given tree (by assuming level of root node is at level 0).

depth of a tree = max level of any node in a given tree.

as tree data structure can grow up to any level and any node can have any number of child nodes, operations on it becomes inefficient, so restrictions can be applied on it to achieve efficiency and hence there are different types of tree.

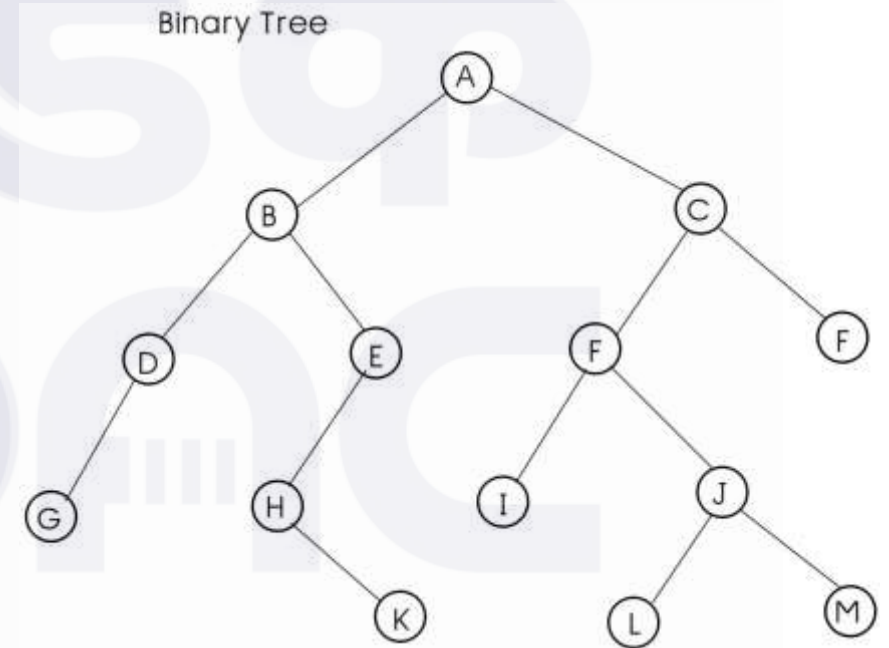
Tree

Binary tree: it is a tree in which each node can have max 2 number of child nodes, i.e. each node can have either 0 OR 1 OR 2 number of child nodes.

OR

Binary tree: it is a set of finite number of elements having three subsets:

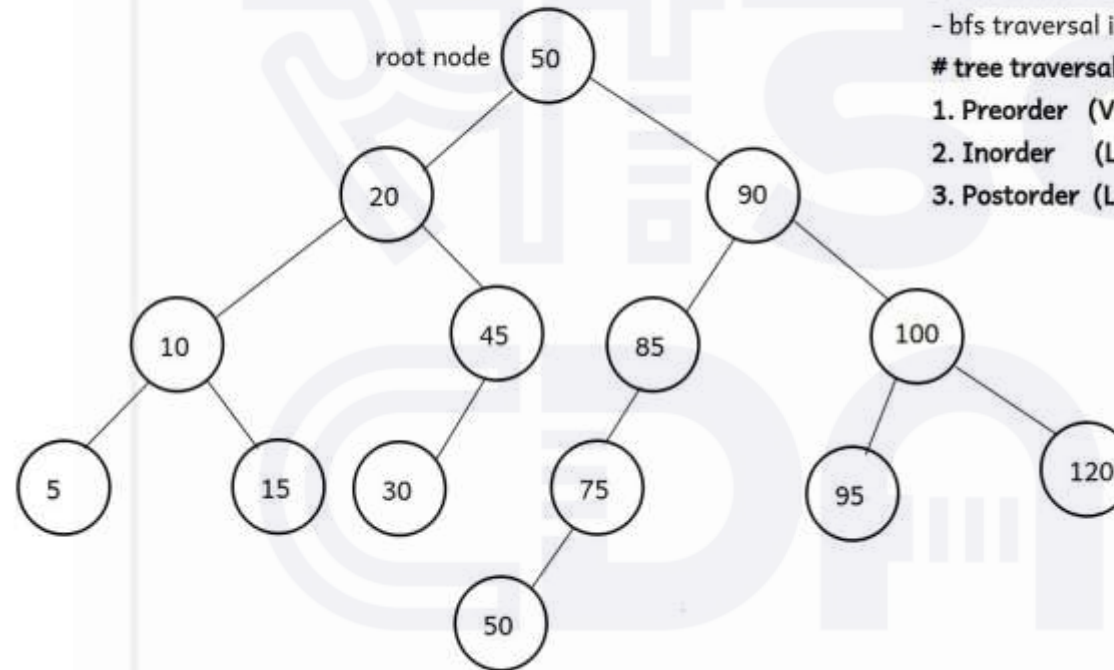
1. root element
2. left subtree (may be empty)
3. right subtree (may be empty)



Tree

Binary Search Tree(BST): it is a binary tree in which left child is always smaller than its parent and right child is always greater than or equal to its parent.

input order of an ele's for BST: 50 20 90 85 10 45 30 100 15 75 95 120 5 50



1. dfs traversal: 50 20 10 5 15 45 30 90 85 75 50 100 95 120

2. bfs traversal: 50 20 90 10 45 85 100 5 15 30 75 95 120 50

- bfs traversal is also called as "levelwise traversal".

tree traversal methods on BST:

1. Preorder (VLR) : 50 20 10 5 15 45 30 90 85 75 50 100 95 120

2. Inorder (LVR): 5 10 15 20 30 45 50 50 75 85 90 95 100 120

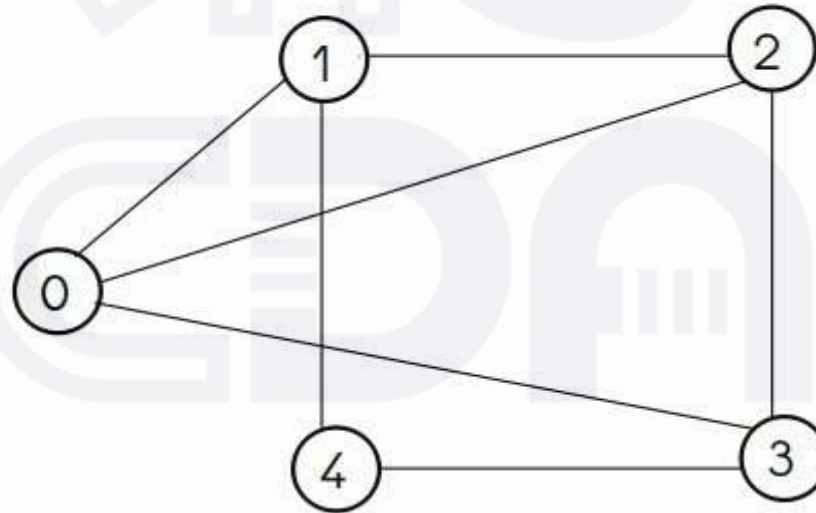
3. Postorder (LRV): 5 15 10 30 45 20 50 75 85 95 120 100 90 50

Graph

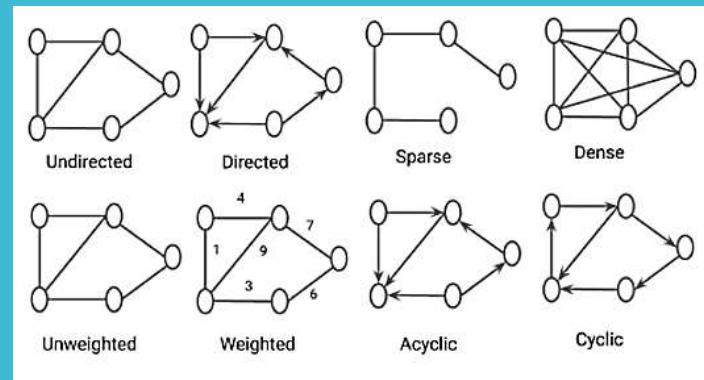
Graph: It is non-linear, advanced data structure, which is a collection of logically related similar and dissimilar type of elements which contains:

- set of finite no. of elements referred as a **vertices**, also called as **nodes**, and
- set of finite no. of ordered/unordered pairs of vertices referred as an **edges**, also called as an **arcs**, whereas it may carry weight/cost/value (cost/weight/value may be -ve).

$G(V,E): V=\{0,1,2,3,4\}; E=\{ (0,1),(0,2),(0,3),(1,2),(1,4),(2,3),(3,4) \}$



Graph



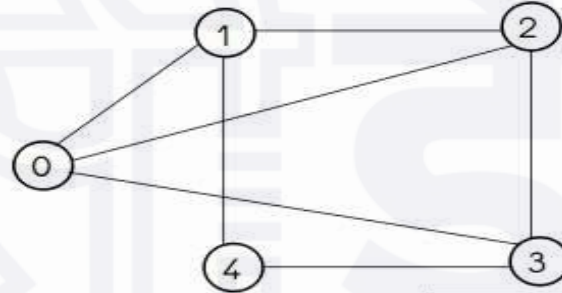
- If there exists a direct edge between two vertices then those vertices are referred as **adjacent vertices** otherwise non-adjacent.
- if we can represent any edge either (u,v) OR (v,u) then it is referred as unordered pair of vertices i.e. **undirected edge**.
- $(u,v) == (v,u) \rightarrow$ unordered pair of vertices $\rightarrow$ undirected edge $\rightarrow$ undirected graph
- if we cannot represent any edge either (u,v) OR (v,u) then it is referred as ordered pair of vertices i.e. **directed edge**.
- $(u,v) != (v,u) \rightarrow$ ordered pair of vertices $\rightarrow$ directed edge $\rightarrow$ directed graph (di-graph).
- **Complete graph:** if all the vertices are adjacent to remaining all vertices in a given graph.
- **Connected vertices:** if path exists between two vertices, then those two vertices are referred as a connected vertices otherwise not-connected.
- **Connected graph:** if any vertex is connected to remaining all vertices in a given graph.

Graph

There are two graph representation methods:

- Adjacency Matrix Representation (2-D Array)
- Adjacency List Representation (Array of Linked Lists)

$G(V,E): V=\{0,1,2,3,4\}; E=\{(0,1),(0,2),(0,3),(1,2),(1,4),(2,3),(3,4)\}$



Adjacency Matrix Representation

	0	1	2	3	4
0	0	1	1	1	0
1	1	0	1	0	1
2	1	1	0	1	0
3	1	0	1	0	1
4	0	1	0	1	0

Adjacency List Representation

